

SnapStream: AWS-Powered Media Upload and Analysis Platform

Project Description:

At a time when digital content creation and media management are rapidly growing, organizations often struggle with storing, processing, and analyzing large volumes of multimedia data efficiently. To address these challenges, the development team created SnapStream — an AWS-Powered Media Upload and Analysis Platform designed to automate and streamline end-to-end media handling.

SnapStream leverages a Flask-based backend for managing user interactions and API integrations, while AWS EC2 provides scalable hosting for the web application. Uploaded media files — including images, audio, and video — are securely stored in Amazon S3, with each upload triggering AWS Lambda for serverless, event-driven processing. The platform integrates AI/ML services such as Amazon Rekognition, Transcribe, and Comprehend to automatically extract metadata, generate transcripts, and perform intelligent content analysis.

All metadata and analysis results are stored in AWS DynamoDB, ensuring fast, reliable, and schema-flexible data retrieval. AWS SNS provides real-time notifications to users about upload and processing status, while AWS IAM roles ensure secure and role-based access to all cloud resources.

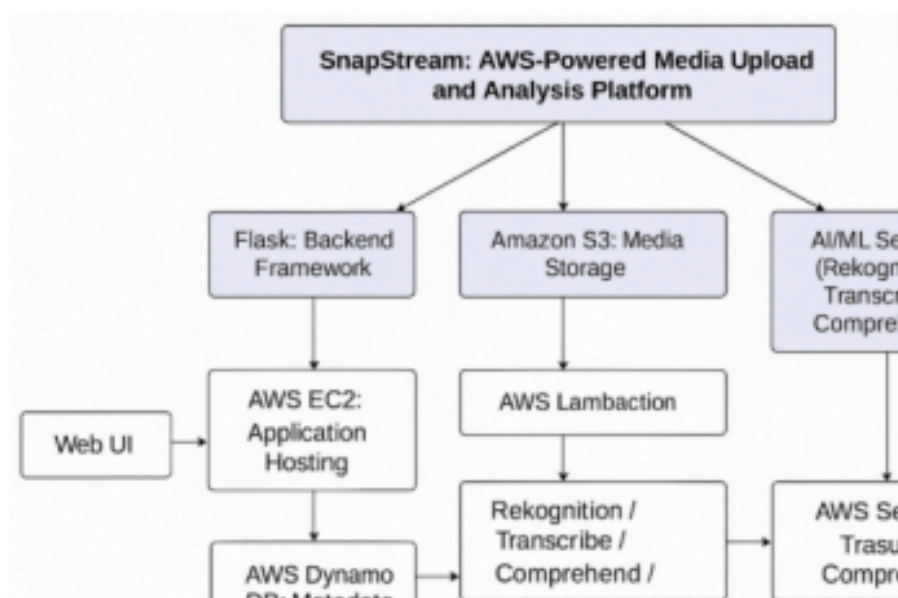
Built with a responsive HTML/CSS frontend, SnapStream delivers a smooth and intuitive user experience, enabling users to upload, manage, and analyze their media effortlessly. This cloud-native platform provides a cost-effective, scalable, and intelligent solution for modern media workflows—reducing manual effort and empowering users with deeper insights into their multimedia content.

Scenario 3: Image Moderation for an E-Learning Platform

The Next-Gen Hospital system allows patients to easily view available doctors, their specialties, and appointment slots. Patients can also manage their bookings, update personal details, or cancel/reschedule appointments through a modern and accessible web interface. DynamoDB ensures quick and secure access to data, while Flask fetches and updates records in real-time. With EC2 hosting and IAM-secured access to AWS resources, the hospital benefits from a secure, scalable, and always-on platform that enhances the patient experience while reducing administrative overhead.

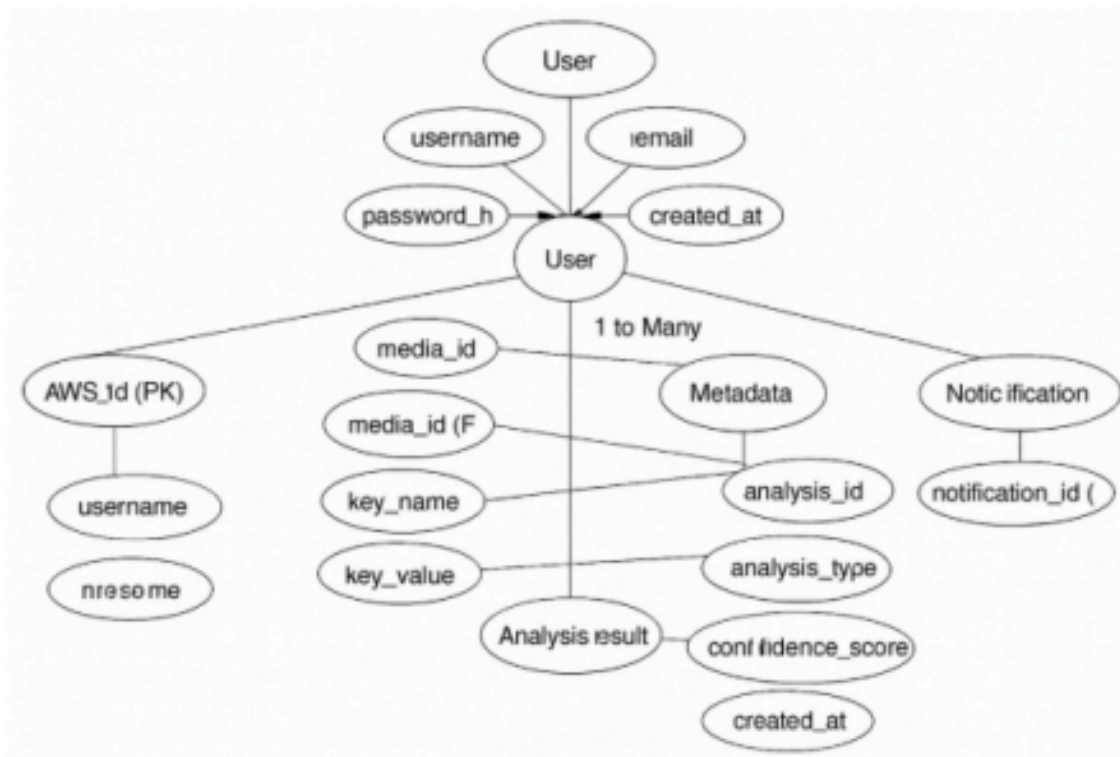
ARCHITECTURE

SnapStream uses a Flask backend hosted on AWS EC2 with Amazon S3 for secure media storage. Uploaded files trigger AWS Lambda for processing using Rekognition, Transcribe, and Comprehend. Metadata is stored in DynamoDB, notifications are sent via SNS, and IAM with CloudWatch ensures security and monitoring.



Entity Relationship (ER)Diagram

SnapStream's ER diagram defines User as the central entity with one-to-many relationships to Media, Metadata, Analysis, and Notification entities. Each media upload stores metadata and analysis results, while notifications track processing updates linked securely to users.



Pre-requisites

1. AWS Account Setup: [AWS Account Setup](#)
2. Understanding IAM: [IAM Overview](#)
3. Amazon EC2 Basics: [EC2 Tutorial](#)
4. DynamoDB Basics: [DynamoDB Introduction](#)
5. Git Version Control: [Git Documentation](#)

Project WorkFlow

1. AWS Account Setup and Login

Activity 1.1: Set up an AWS account if not already done.

Activity 1.2: Log in to the AWS Management Console

2. DynamoDB Database Creation and Setup

Activity 2.1: Create a DynamoDB Table.

Activity 2.2: Configure Attributes for User Data and Book Requests.

3. SNS Notification Setup

Activity 3.1: Create SNS topics for sending email notifications to users and library staff

Activity 3.2: Subscribe users and staff to relevant SNS topics to receive real-time notifications.

4. Backend Development and Application Setup

Activity 4.1: Develop the Backend Using Flask.

Activity 4.2: Integrate AWS Services Using boto3.

5. EC2 Instance Setup

Activity 5.1: Launch an EC2 instance to host the Flask application.

Activity 5.2: Configure security groups for HTTP, and SSH access.

6. Deployment on EC2

Activity 6.1: Upload Flask Files

Activity 6.2: Run the Flask App

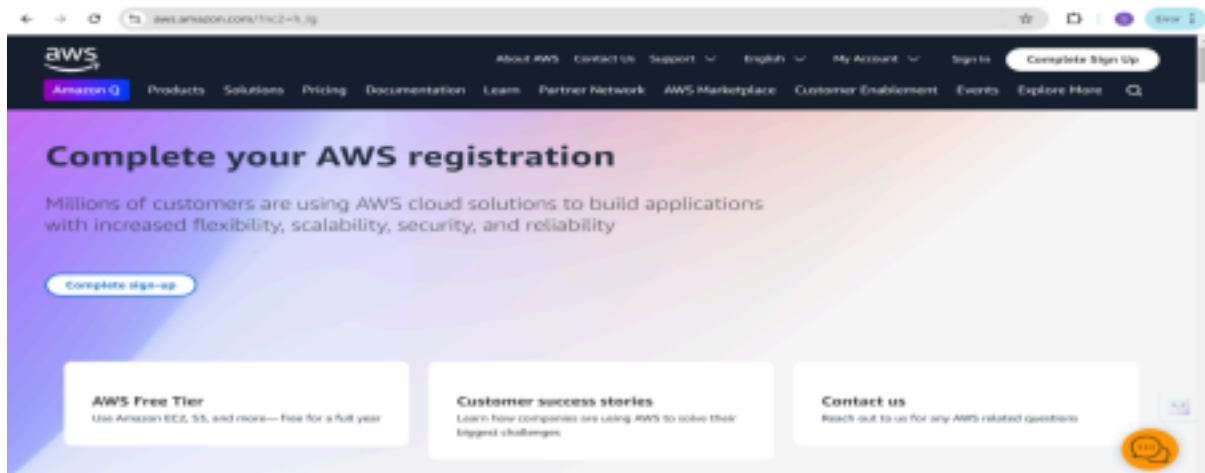
7. Testing and Deployment

Activity 7.1: Conduct functional testing to verify user registration, login, book requests, and notifications.

Milestone 1: AWS Account Setup and Login

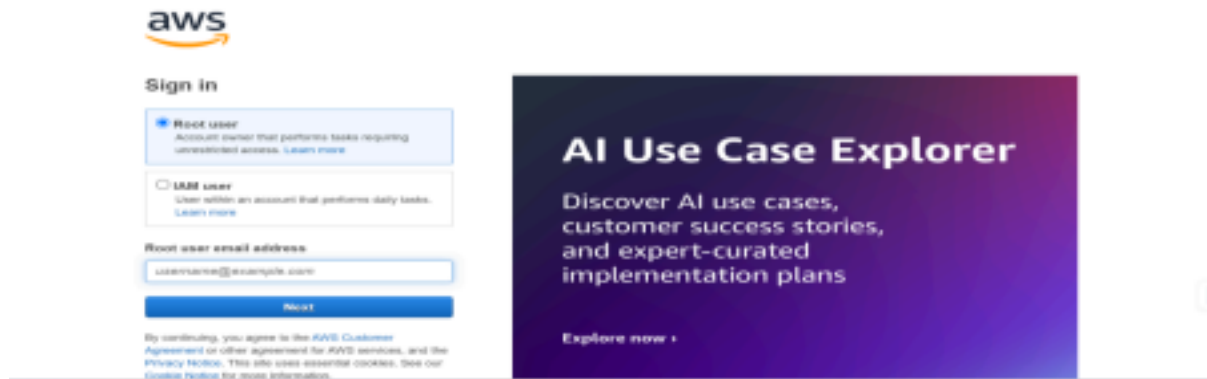
Activity 1.1: Set up an AWS account if not already done.

- Sign up for an AWS account and configure billing settings.



Activity 1.2: Log in to the AWS Management Console

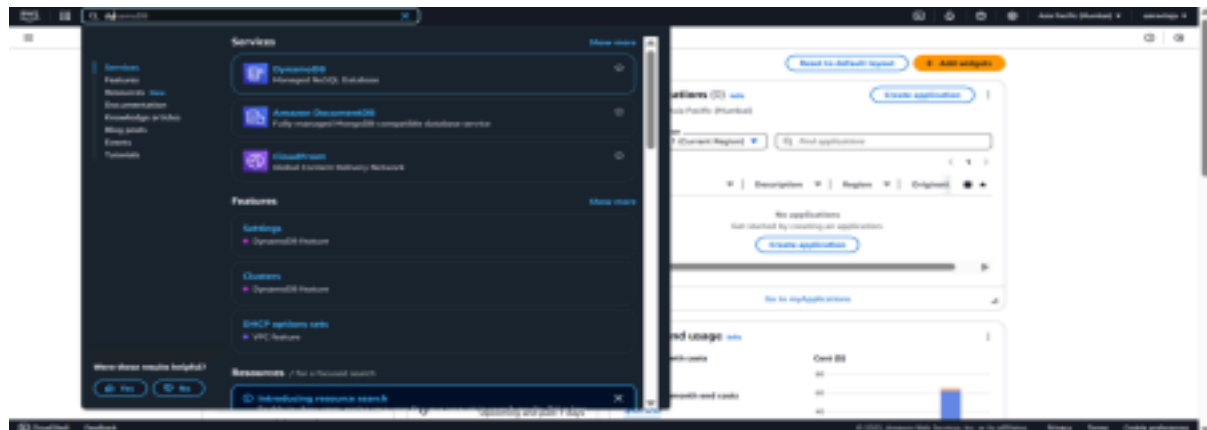
- After setting up your account, log in to the [AWS Management Console](#).

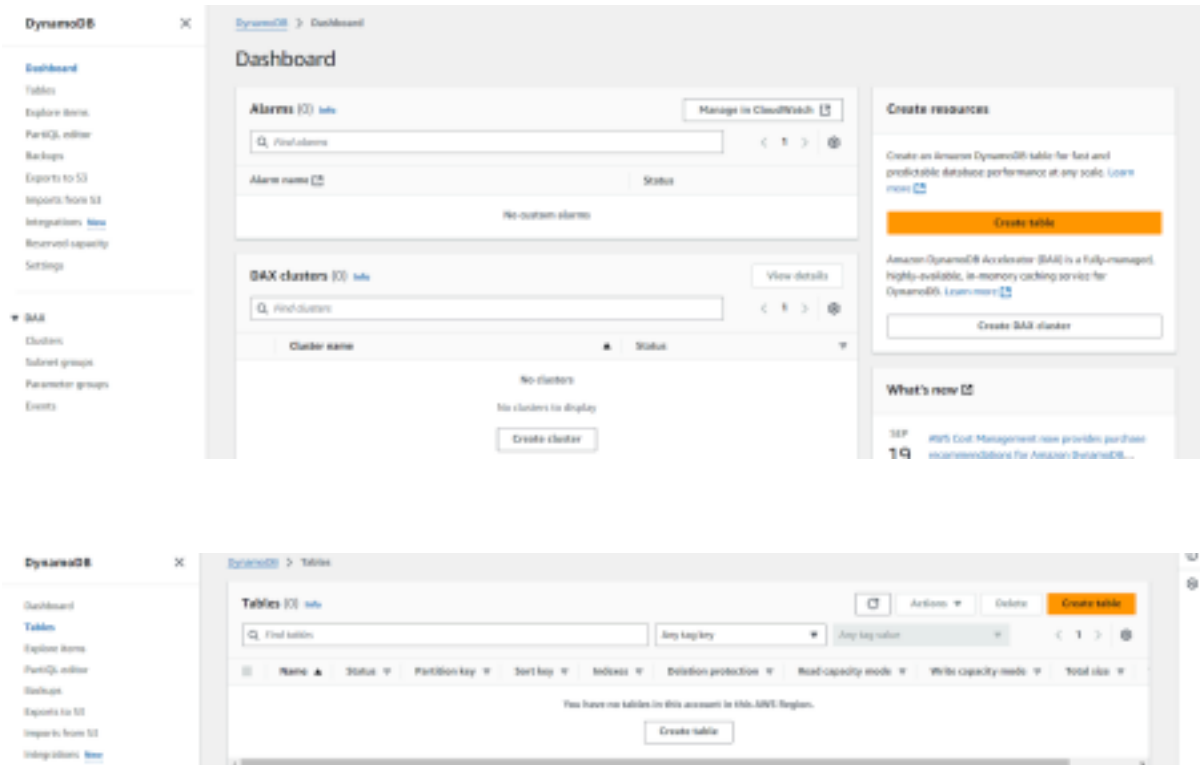


Milestone 2: DynamoDB Database Creation and Setup

Activity 2.1: Navigate to the DynamoDB

- In the AWS Console, navigate to DynamoDB and click on create tables.

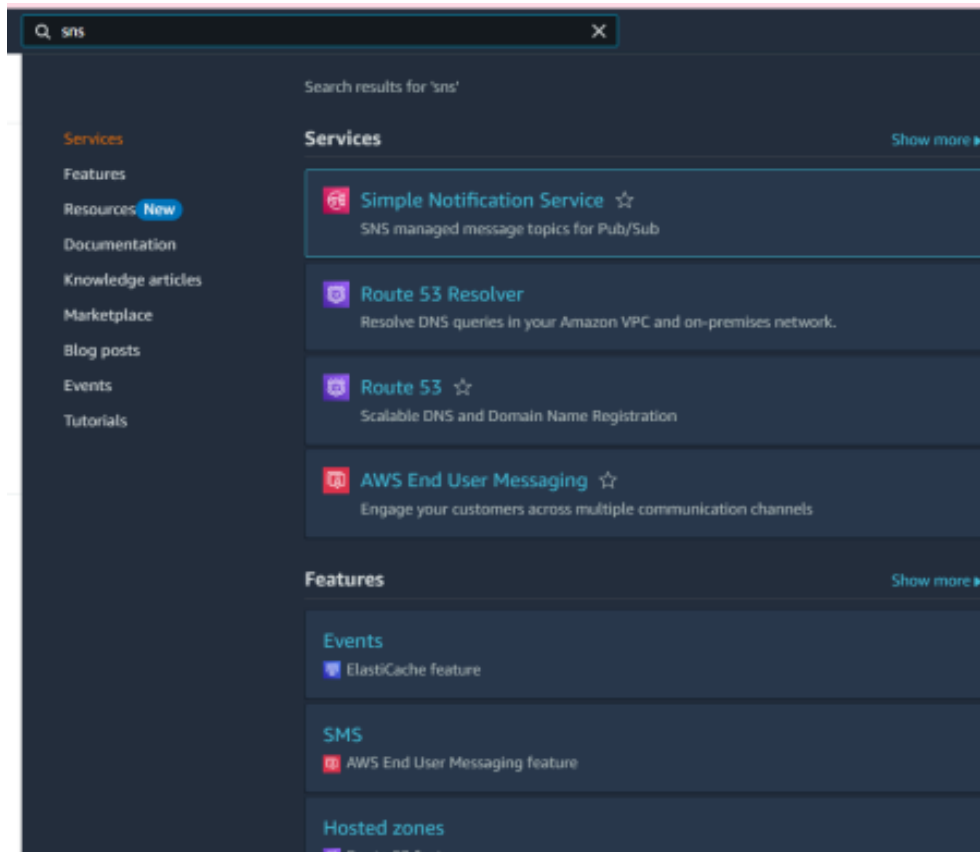




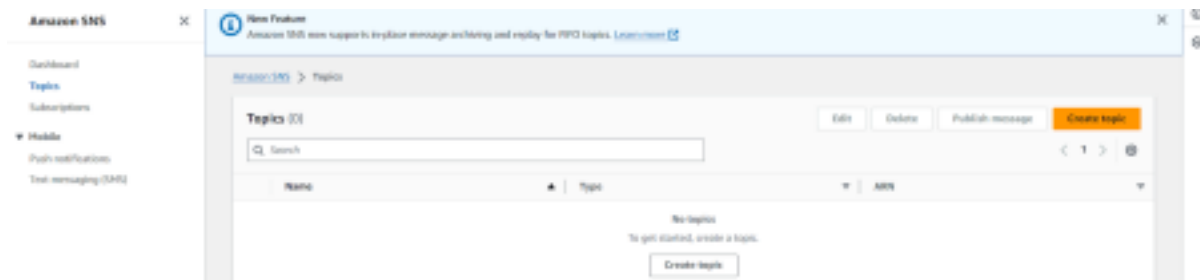
Milestone 3: SNS Notification Setup

Activity 3.1: Create SNS topics for sending email notifications to users and library staff.

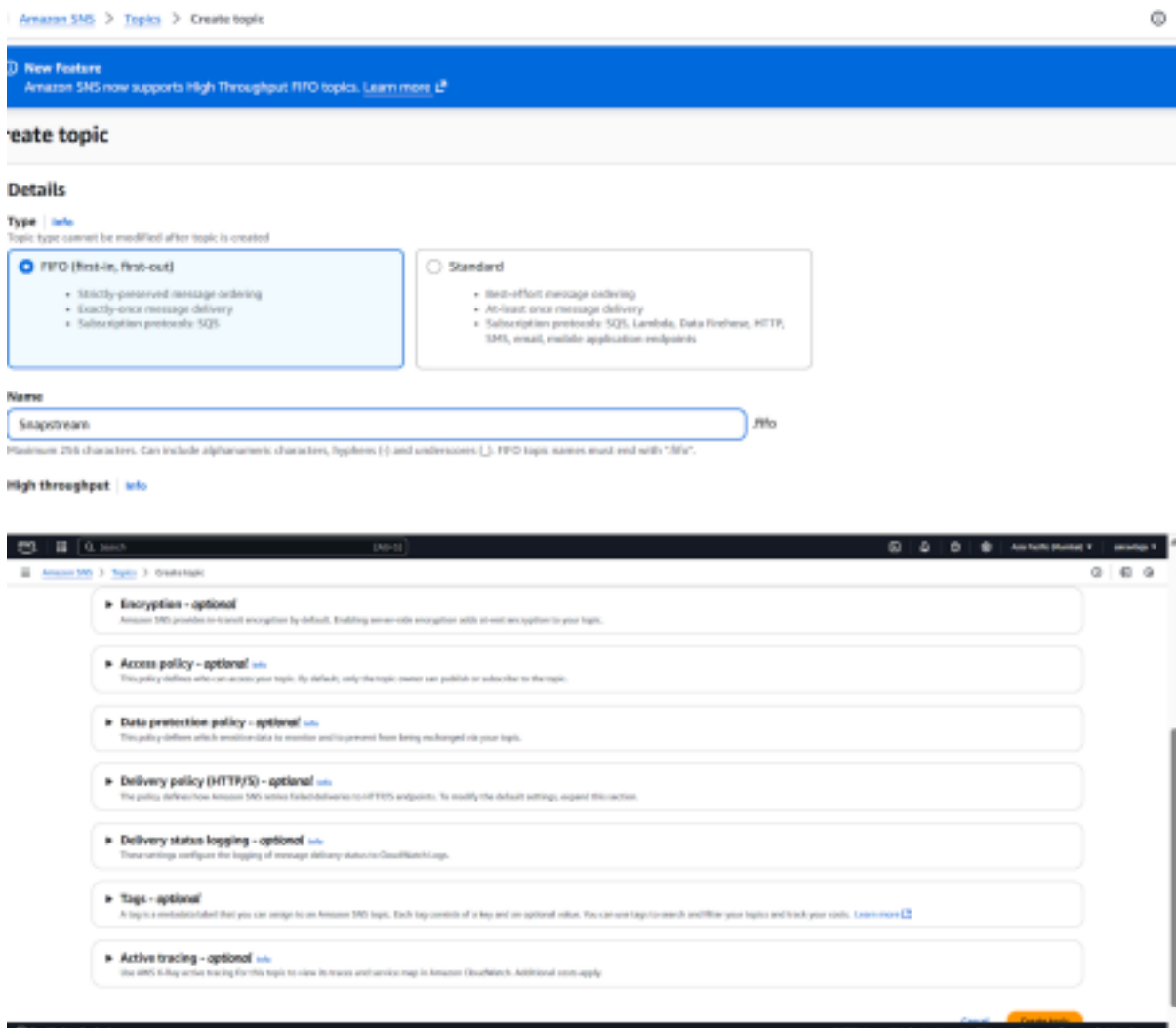
- In the AWS Console, search for SNS and navigate to the SNS Dashboard.



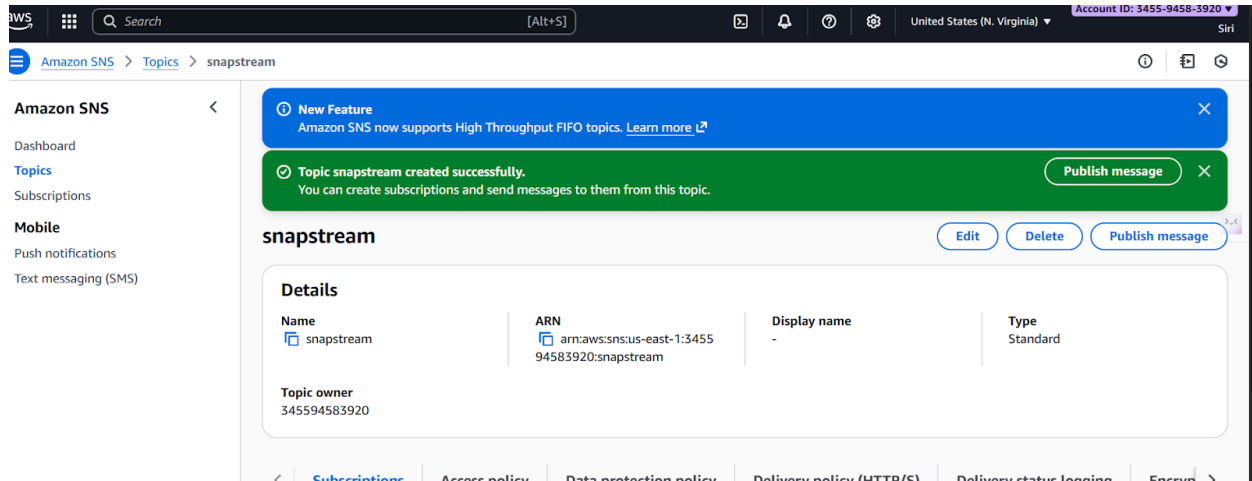
- Click on **Create Topic** and choose a name for the topic.



Choose Standard type for general notification use cases and Click on Create Topic



- Configure the SNS topic and note down the **Topic ARN**.



Activity 3.2: Subscribe users and staff to relevant SNS topics to receive real-time notifications.

- Subscribe users (or admin staff) to this topic via Email. When a book request is made, notifications will be sent to the subscribed emails.

Create subscription

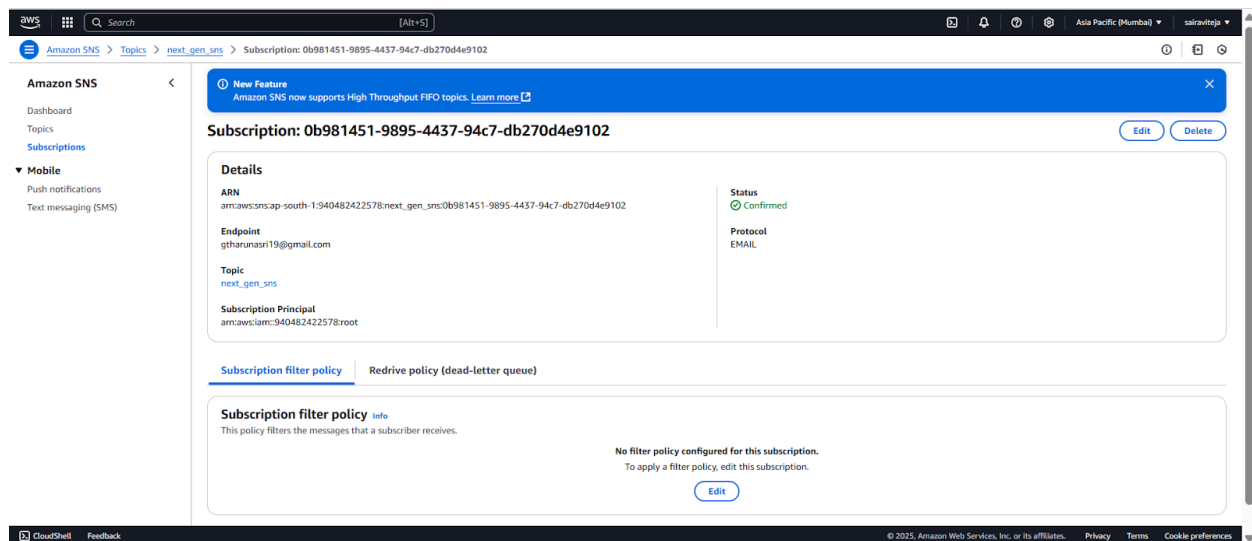
Details

Topic ARN

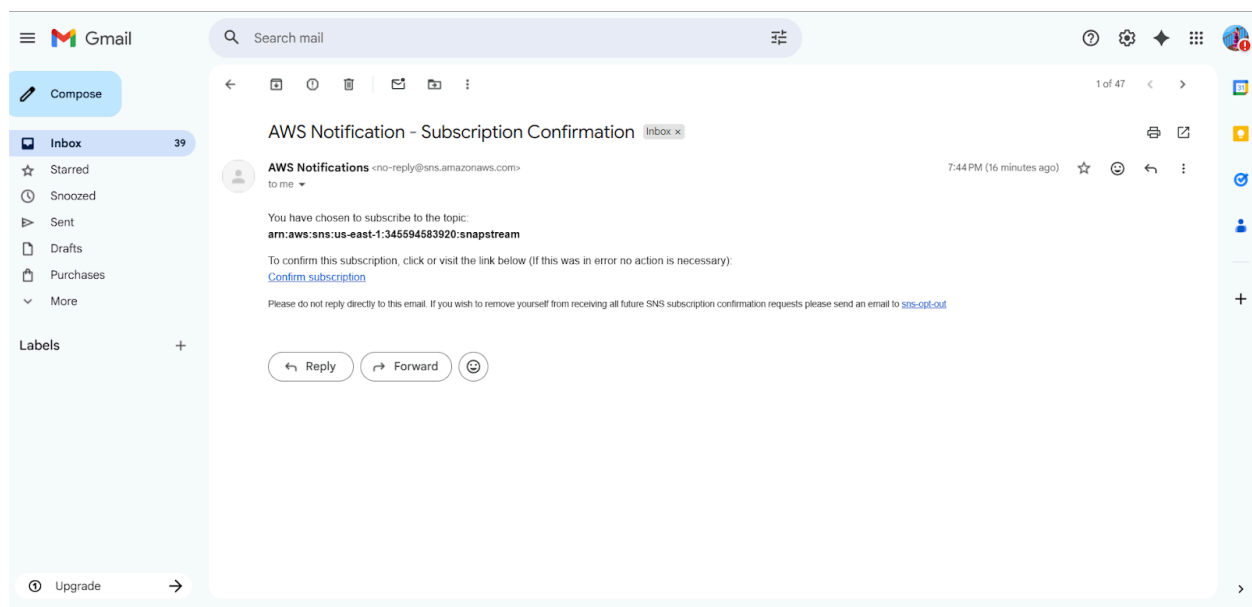
Protocol
The type of endpoint to subscribe

Endpoint
An email address that can receive notifications from Amazon SNS.

After your subscription is created, you must confirm it. [Info](#)



- After subscription request for the mail confirmation
- Navigate to the subscribed Email account and Click on the confirm subscription in the AWS Notification- Subscription Confirmation mail.



Successfully done with the SNS mail subscription and setup, now store the ARN link



Simple Notification Service

Subscription confirmed!

You have successfully subscribed.

Your subscription's id is:

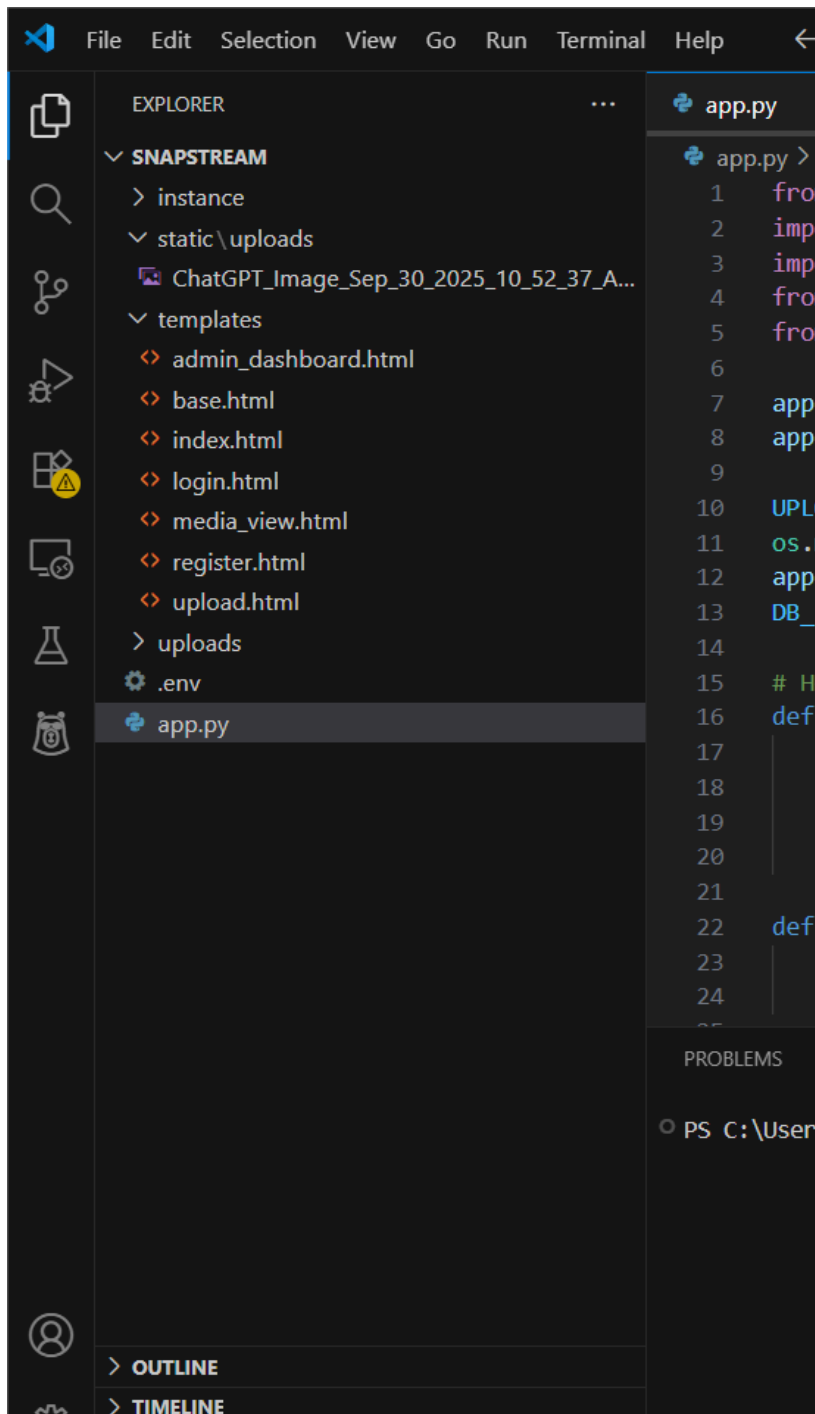
arn:aws:sns:ap-south-1:940482422578:next_gen_sns:0b981451-9895-4437-94c7-db270d4e9102

If it was not your intention to subscribe, [click here to unsubscribe](#).

Milestone 4: Backend Development and Application Setup

Activity 4.1: Develop the backend using Flask

- File Explorer Structure



Description of the code :

- Flask App Initialization

```
aws_app.py > get_tables
1  from flask import Flask, request, jsonify
2  from flask_cors import CORS
3  import boto3
4  import uuid
5  import os
6  from datetime import datetime
7  from botocore.exceptions import ClientError
8
9  app = Flask(__name__)
10 CORS(app)
11
12  # --- AWS Configuration ---
13  REGION = os.getenv("AWS_REGION", "us-east-1")
14  SNS_TOPIC_ARN = os.getenv(
15      "SNS_TOPIC_ARN",
16      "arn:aws:sns:us-east-1:000000000000:local-topic"
17  )
18
19  # ----- AWS HELPERS (IMPORTANT FOR MOTO) -----
20
21  def get_dynamodb():
22      | return boto3.resource("dynamodb", region_name=REGION)
23
24  def get_sns():
25      | return boto3.client("sns", region_name=REGION)
26
27  def get_tables():
28      | dynamodb = get_dynamodb()
29      | return [
30      |     dynamodb.Table("Users"),
31      |     dynamodb.Table("Doctors"),
32      |     dynamodb.Table("Appointments")
```

```

aws_app.py > get_tables
27 def get_tables():
33     ]
34
35 def send_sns_notification(subject, message):
36     try:
37         sns = get_sns()
38         sns.publish(
39             TopicArn=SNS_TOPIC_ARN,
40             Subject=subject,
41             Message=message
42         )
43     except ClientError as e:
44         print(f"SNS Error: {e}")
45
46 # ----- API ROUTES -----
47
48 @app.route("/api/status", methods=["GET"])
49 def status():
50     return jsonify({
51         "status": "online",
52         "engine": "DynamoDB",
53         "environment": "Cloud/EC2" if os.getenv("AWS_EXECUTION_ENV") else "Local"
54     }), 200
55
56 @app.route("/api/signup", methods=["POST"])
57 def signup():
58     users_table, _, _ = get_tables()
59     data = request.get_json()
60     email = data["email"].lower()
61
62     try:
63         response = users_table.get_item(Key={"email": email})

```

aws_app.py > get_tables

```
57 def signup():
63     response = users_table.get_item(Key={"email": email})
64     if "Item" in response:
65         return jsonify({"success": False, "message": "Email already exists"}), 409
66
67     user_id = f"USR-{uuid.uuid4().hex[:6].upper()}"
68     user_role = data.get("role", "PATIENT").upper()
69
70     users_table.put_item(Item={
71         "email": email,
72         "id": user_id,
73         "username": data["name"],
74         "password": data["password"],
75         "role": user_role,
76         "securityQuestion": data["securityQuestion"],
77         "securityAnswer": data["securityAnswer"].lower()
78     })
79
80     send_sns_notification(
81         "New User Signup",
82         f"{email} registered as {user_role}"
83     )
84
85     return jsonify({
86         "success": True,
87         "user": {
88             "id": user_id,
89             "email": email,
90             "role": user_role
91         }
92     }), 201
93
```



```
app.py • aws_app.py M •
aws_app.py > get_tables
57 def signup():
58     jsonify({"error": str(e)}), 201
59
60 except Exception as e:
61     return jsonify({"error": str(e)}), 500
62
63 @app.route("/api/login", methods=["POST"])
64 def login():
65     users_table, _, _ = get_tables()
66     data = request.get_json()
67
68     email = data.get("email", "").lower()
69     password = data.get("password", "")
70
71     try:
72         response = users_table.get_item(Key={"email": email})
73         if "Item" in response and response["Item"]["password"] == password:
74             user = response["Item"]
75             return jsonify({"success": True, "user": user}), 200
76
77         return jsonify({"success": False}), 401
78
79     except Exception as e:
80         return jsonify({"error": str(e)}), 500
81
82 @app.route("/api/appointments", methods=["POST"])
83 def create_appointment():
84     _, _, appointments_table = get_tables()
85     data = request.get_json()
86
87     appointment_id = f"APP-{uuid.uuid4().hex[:6].upper()}"
88
89     appointments_table.put_item(Item={
90         "appointment_id": appointment_id,
91         "email": data.get("email", ""),
92         "password": data.get("password", "")
93     })
94     return jsonify({"success": True, "appointment_id": appointment_id}), 201
95
96 except Exception as e:
97     return jsonify({"error": str(e)}), 500
98
99 if __name__ == '__main__':
100     app.run(debug=True)
```

```
app.py  aws_app.py M
aws_app.py > get_tables
117 def create_appointment():
123     appointments_table.put_item(Item={
124         "id": appointment_id,
125         "patientEmail": data["patientEmail"],
126         "doctorName": data["doctorName"],
127         "date": data["date"],
128         "time": data["time"],
129         "status": "pending",
130         "timestamp": datetime.utcnow().isoformat()
131     })
132
133     send_sns_notification(
134         "Appointment Booked",
135         f"{data['patientEmail']} booked Dr. {data['doctorName']}"
136     )
137
138     return jsonify({"success": True, "id": appointment_id}), 201
139
140 if __name__ == "__main__":
141     app.run(host="0.0.0.0", port=5000, debug=True)
142
```

Milestone 5: EC2 Instance Setup

- Note: Load your Flask app and Html files into GitHub repository.



karunamanirathnam / NEXUSCARE

<> Code

Issues

Pull requests

Actions

Projects

Security



NEXUSCARE

Public

main

1 Branch 0 Tags

Search



karunamanirathnam Update aws_app.py



components

AuthPortal.tsx



services

AuthPortal.tsx



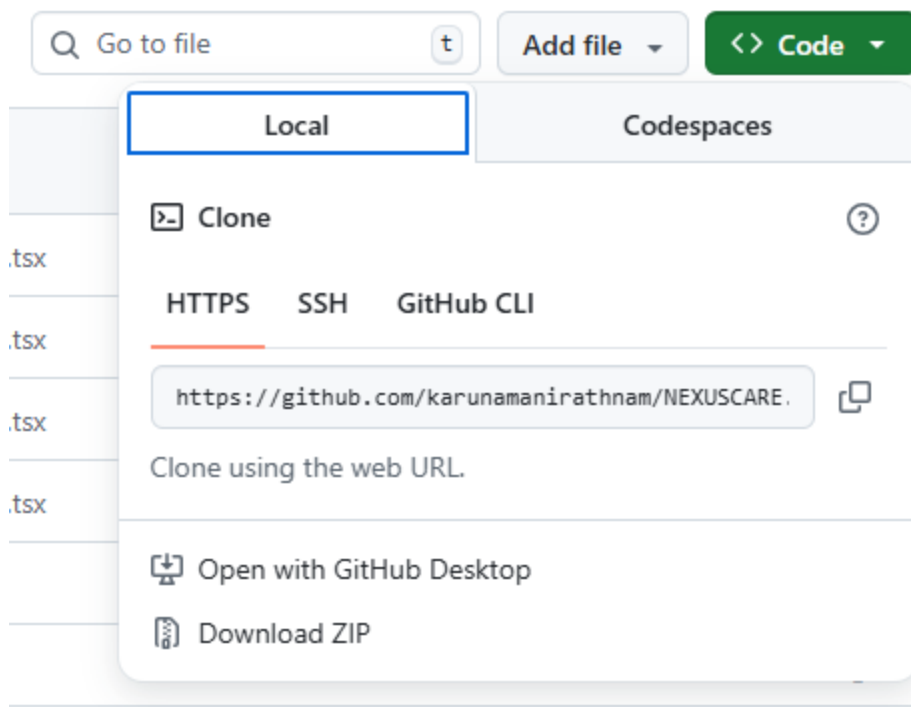
.gitignore

AuthPortal.tsx



App.tsx

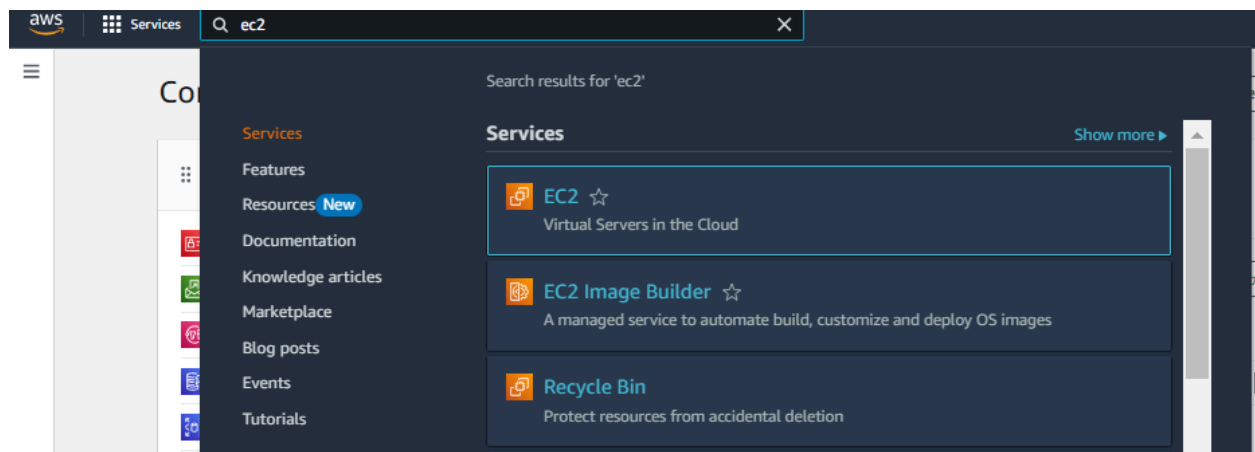
AuthPortal.tsx



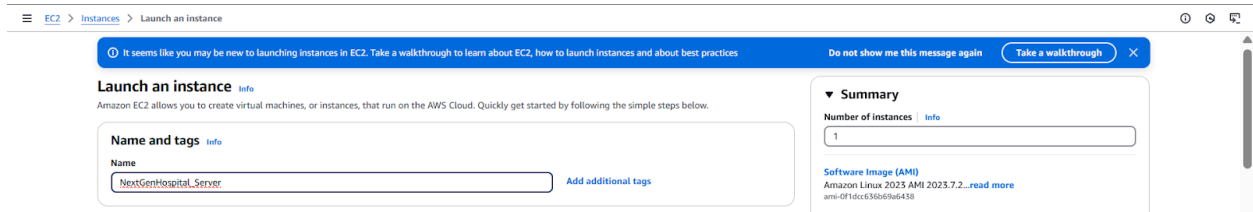
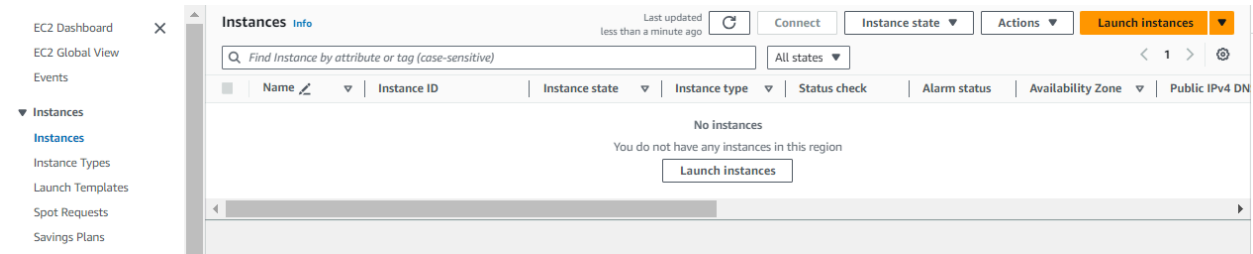
Activity 5.1: Launch an EC2 instance to host the Flask application.

- **Launch EC2 Instance**

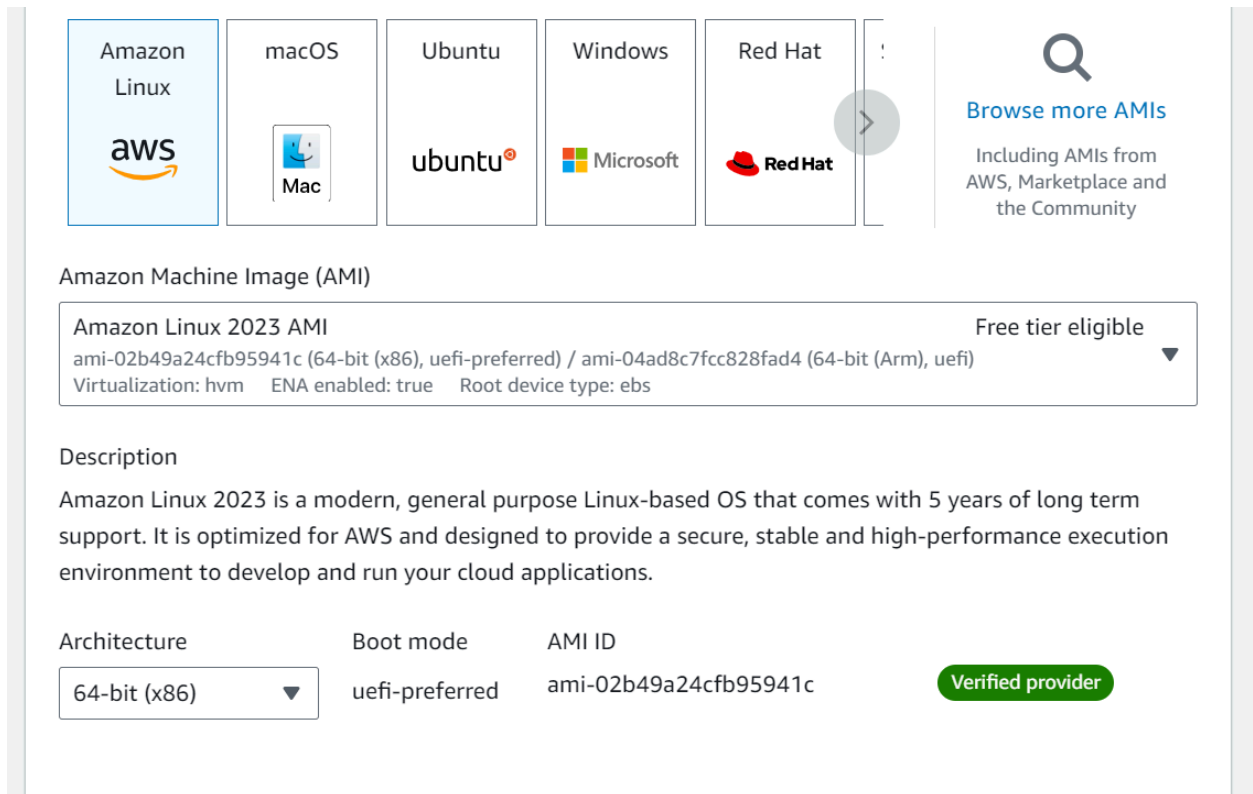
- In the AWS Console, navigate to EC2 and launch a new instance.



- Click on Launch instance to launch EC2 instance



- Choose Amazon Linux 2 or Ubuntu as the AMI and t2.micro as the instance type (free-tier eligible).



- Create and download the key pair for Server access.

▼ Instance type [Info](#) | [Get advice](#)

Instance type

t2.micro

Free tier eligible

Family: t2 1 vCPU 1 GiB Memory Current generation: true
On-Demand Linux base pricing: 0.0124 USD per Hour
On-Demand Windows base pricing: 0.017 USD per Hour
On-Demand RHEL base pricing: 0.0268 USD per Hour
On-Demand SUSE base pricing: 0.0124 USD per Hour

☒ All generations

[Compare instance types](#)

Additional costs apply for AMIs with pre-installed software

▼ Key pair (login) [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - *required*

Select

[Create new key pair](#)

[EC2](#) > [Key pairs](#) > Create key pair

Create key pair [Info](#)

Key pair

A key pair, consisting of a private key and a public key, is a set of security credentials that you use to prove your identity when connecting to an instance.

Name

Snapstream

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type [Info](#)

☒ RSA

☐ ED25519

Private key file format

☒ .pem

For use with OpenSSH

☐ .ppk

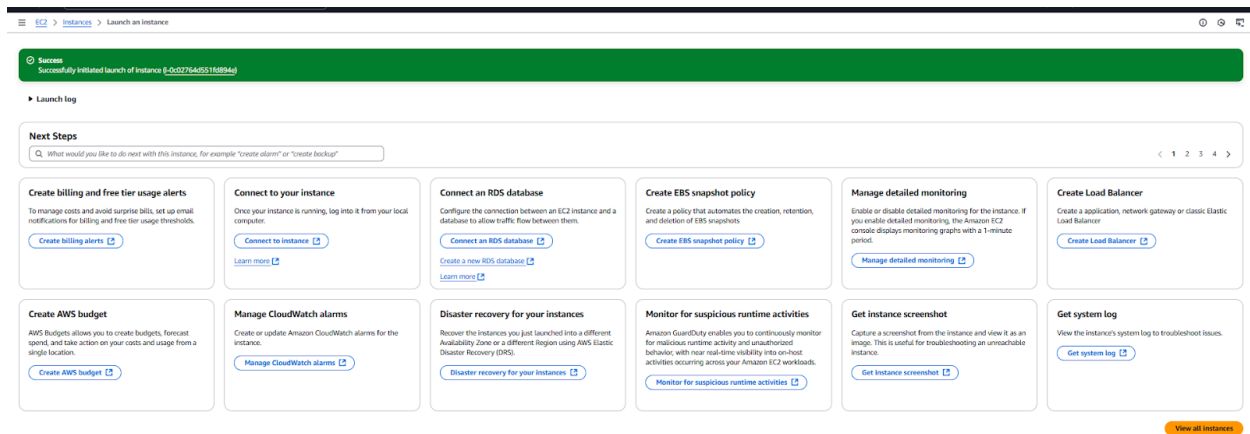
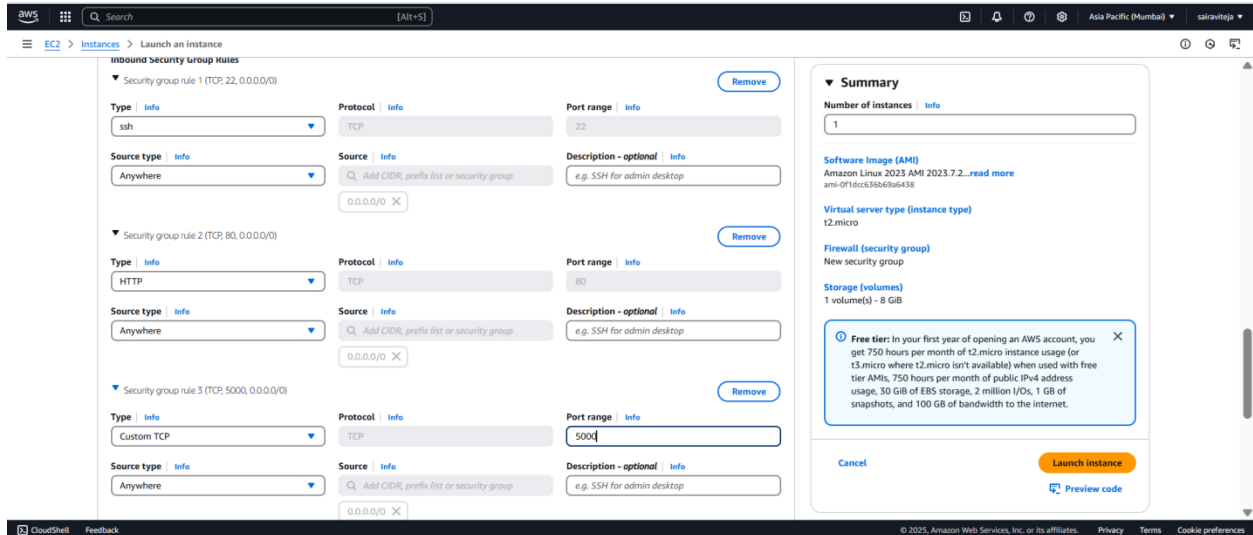
For use with PuTTY

Tags - *optional*

No tags associated with the resource.

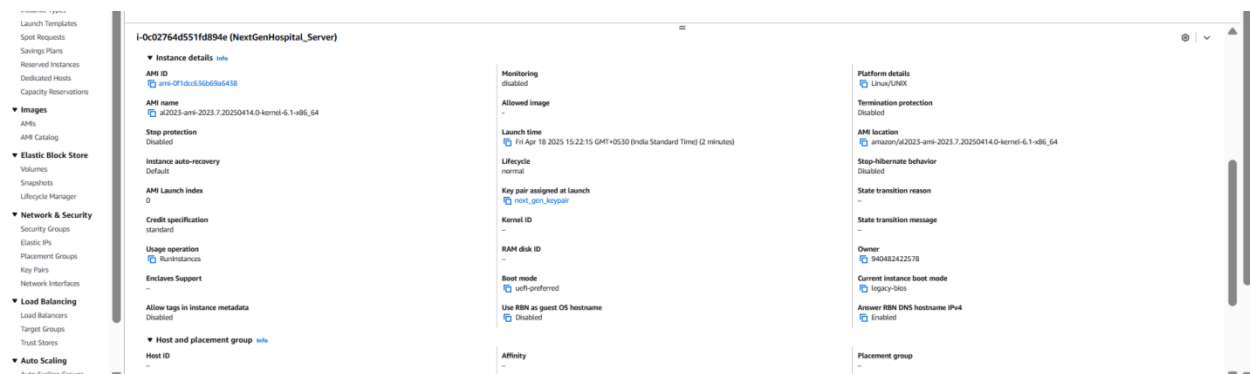
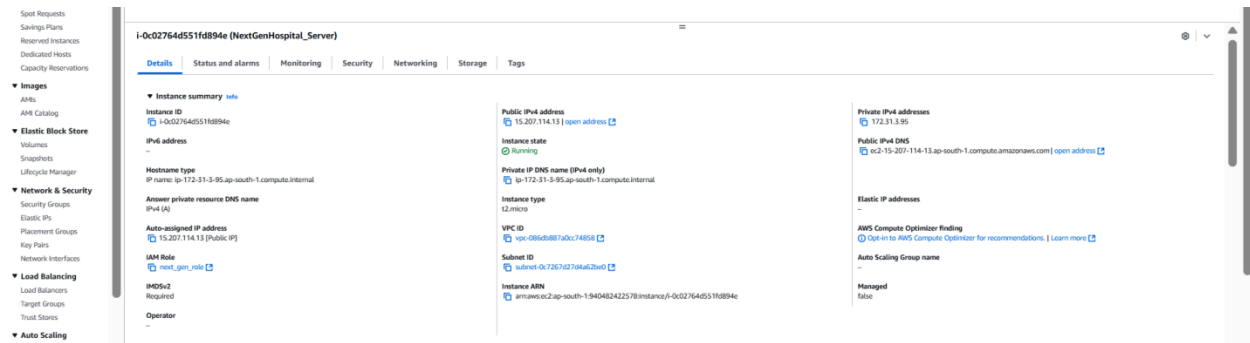
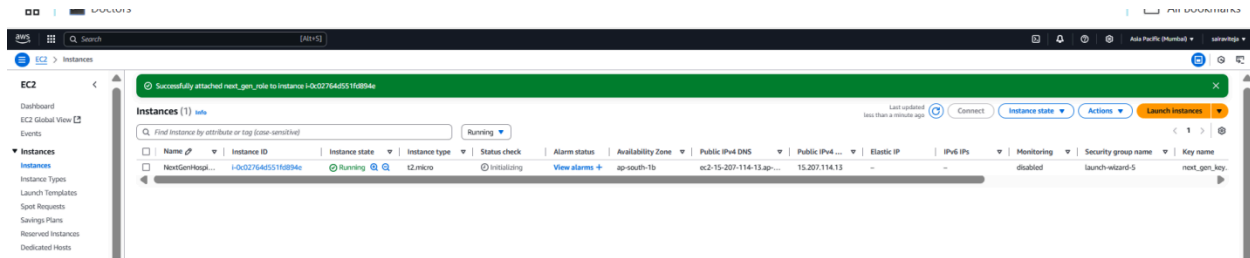
[Add new tag](#)

Activity 5.2: Configure security groups for HTTP, and SSH access.

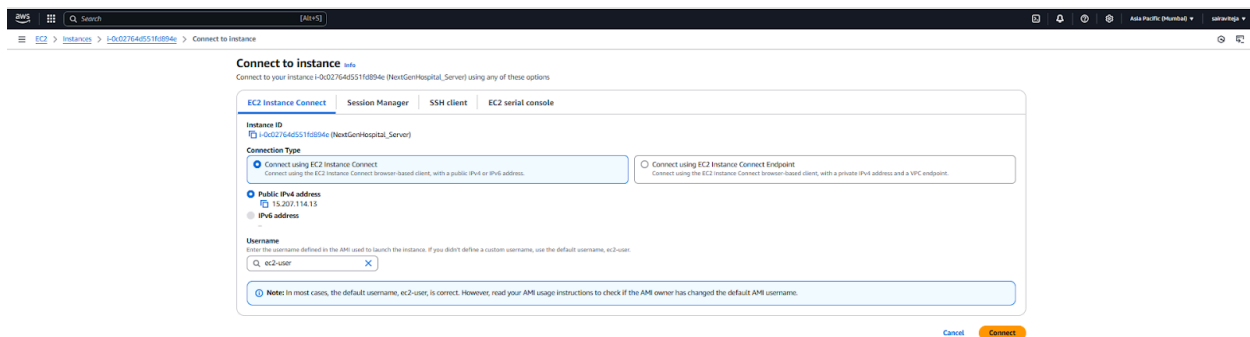


- To connect to EC2 using EC2 Instance Connect, start by ensuring that an IAM role is attached to your EC2 instance. You can do this by selecting your instance, clicking on Actions, then navigating to Security and

- navigate to the EC2 section in the AWS Management Console. Select the EC2 instance you wish to connect to. At the top of the EC2 Dashboard, click the Connect button. From the connection methods presented, choose EC2 Instance Connect. Finally, click Connect again, and a new browser-based terminal will open, allowing you to access your EC2 instance directly from your browser.



- Now connect the EC2 with the files





Milestone 6: Deployment on EC2

Activity 6.1: Install Software on the EC2 Instance

Install Python3, Flask, and Git:

On Amazon Linux 2:

```
sudo yum update -y
```

```
sudo yum install python3 git
```

```
sudo pip3 install flask boto3
```

Verify Installations:

```
flask --version
```

```
git --version
```

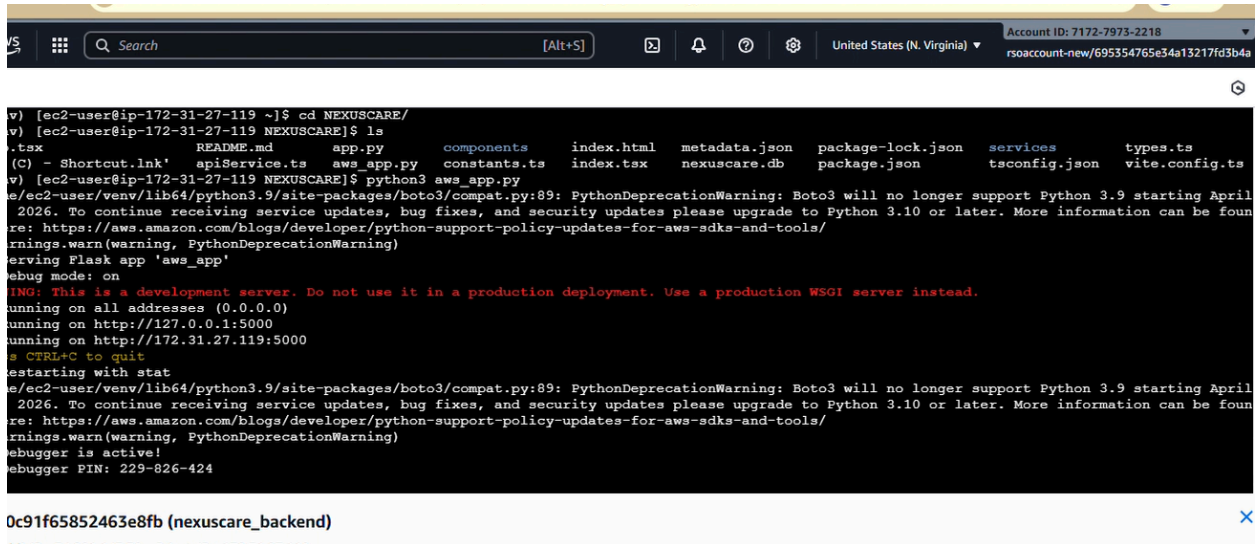
Activity 6.2: Clone Your Flask Project from GitHub

Clone your project repository from GitHub into the EC2 instance using Git.

Verify the Flask app is running:

http://your-ec2-public-ip

- Run the Flask app on the EC2 instance



```
v) [ec2-user@ip-172-31-27-119 ~]$ cd NEXUSCARE/
v) [ec2-user@ip-172-31-27-119 NEXUSCARE]$ ls
.tsx      README.md  app.py      components  index.html  metadata.json  package-lock.json  services  types.ts
(c) - Shortcut.lnk'  apiService.ts  aws_app.py  constants.ts  index.tsx    nexuscare.db  package.json        tsconfig.json  vite.config.ts
v) [ec2-user@ip-172-31-27-119 NEXUSCARE]$ python3 aws_app.py
e/ec2-user/venv/lib64/python3.9/site-packages/boto3/compat.py:89: PythonDeprecationWarning: Boto3 will no longer support Python 3.9 starting April
2026. To continue receiving service updates, bug fixes, and security updates please upgrade to Python 3.10 or later. More information can be found
re: https://aws.amazon.com/blogs/developer/python-support-policy-updates-for-aws-sdks-and-tools/
rnings.warn(warning, PythonDeprecationWarning)
erving Flask app 'aws_app'
ebug mode: on
ING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
unning on all addresses (0.0.0.0)
unning on http://127.0.0.1:5000
unning on http://172.31.27.119:5000
s CTRL+C to quit
estarting with stat
e/ec2-user/venv/lib64/python3.9/site-packages/boto3/compat.py:89: PythonDeprecationWarning: Boto3 will no longer support Python 3.9 starting April
2026. To continue receiving service updates, bug fixes, and security updates please upgrade to Python 3.10 or later. More information can be found
re: https://aws.amazon.com/blogs/developer/python-support-policy-updates-for-aws-sdks-and-tools/
rnings.warn(warning, PythonDeprecationWarning)
ebugger is active!
ebugger PIN: 229-826-424

0c91f65852463e8fb (nexuscare_backend)
```

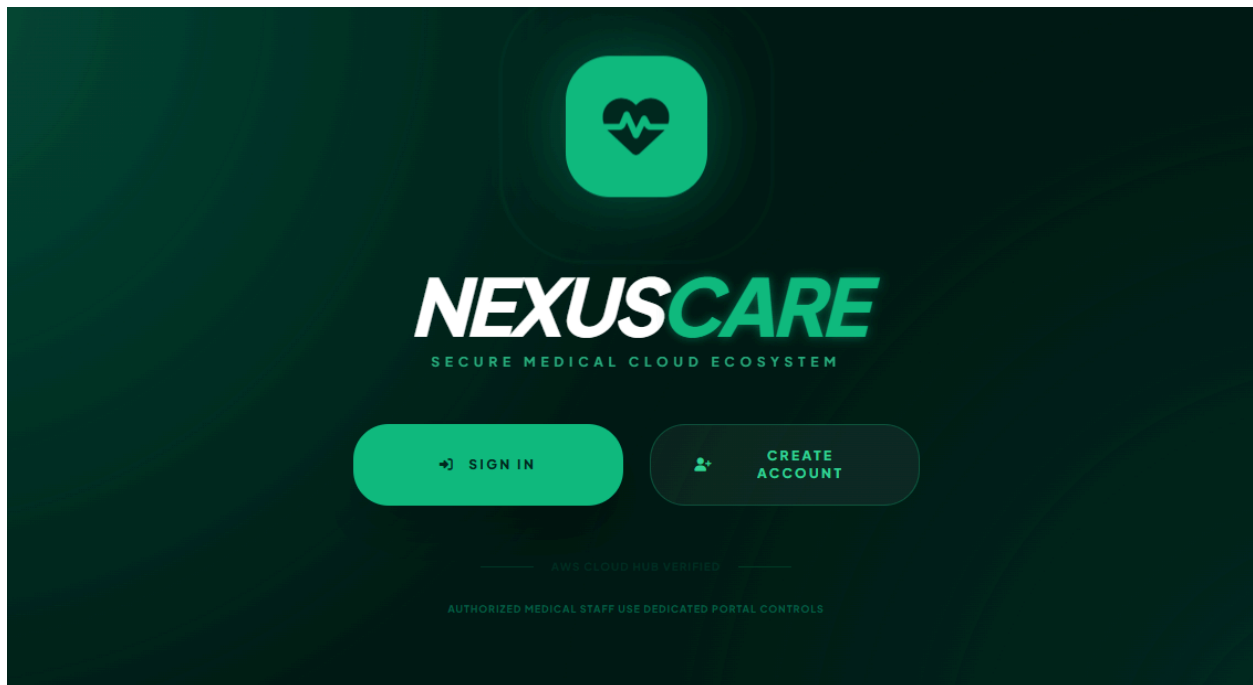
Access the website through:

PublicIPs: http://98.89.34.16:5000/

Milestone 7: Testing and Deployment

Activity 7.1: Conduct functional testing to verify user registration, login, book requests, and notifications

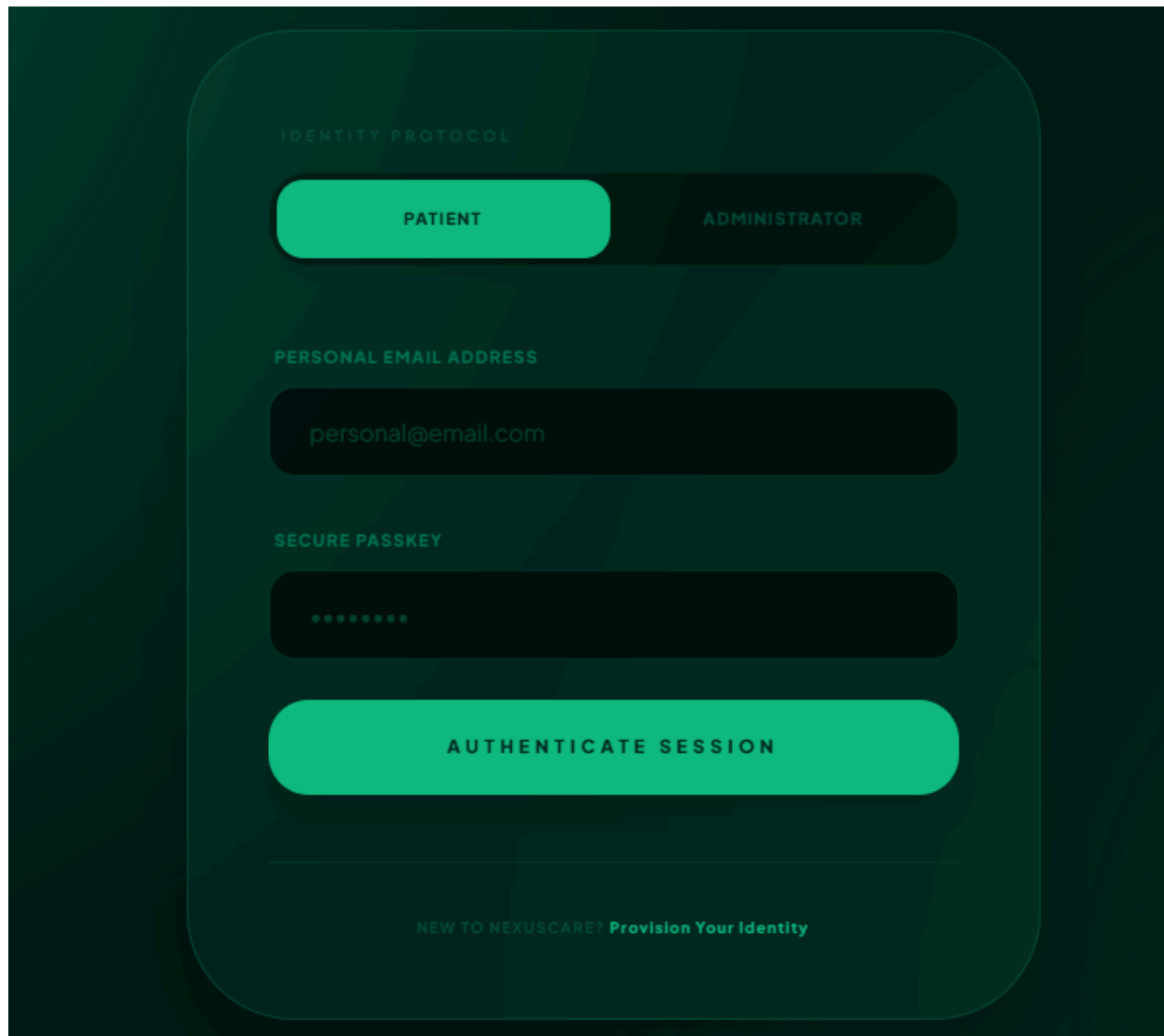
Index page :



Register Page:

The image shows a registration page for NexusCare. At the top center is the "NEXUSCARE" logo, with "NEXUS" in white and "CARE" in light blue. Below the logo is the tagline "SECURE PATIENT IDENTITY HUB" in a smaller, all-caps font. Below the tagline is a section titled "IDENTITY PROTOCOL" in a small, all-caps font. Under this title are two buttons: a blue button with the text "PATIENT" and a white button with the text "ADMINISTRATOR". Below these buttons are three input fields, each with a label above it: "LEGAL NAME" with a placeholder "Patient Full Name", "PERSONAL EMAIL ADDRESS" with a placeholder "personal@email.com", and "SECURE PASSKEY" with a placeholder consisting of eight dots. The background is a dark blue gradient with faint, abstract shapes.

Login Page:



A login page with a dark teal background and a central white rounded rectangle containing the login form. The form includes a role selector with 'PATIENT' and 'ADMINISTRATOR' options, a text input for 'PERSONAL EMAIL ADDRESS' containing 'personal@email.com', a password input for 'SECURE PASKEY' with masked characters, a large 'AUTHENTICATE SESSION' button, and a link for new users at the bottom.

IDENTITY PROTOCOL

PATIENT ADMINISTRATOR

PERSONAL EMAIL ADDRESS

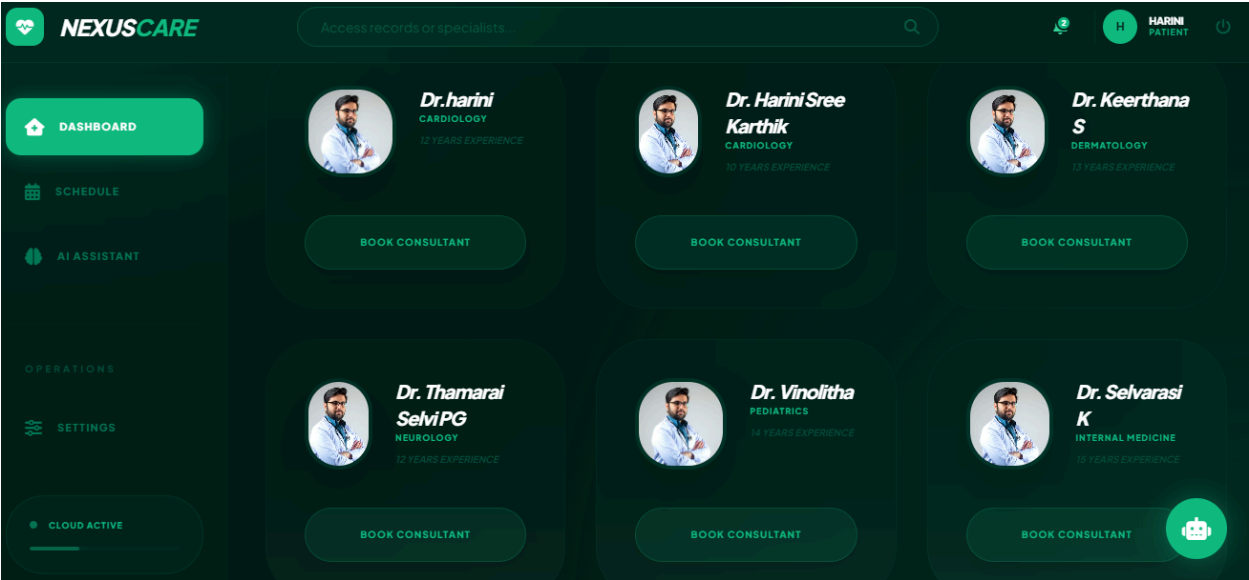
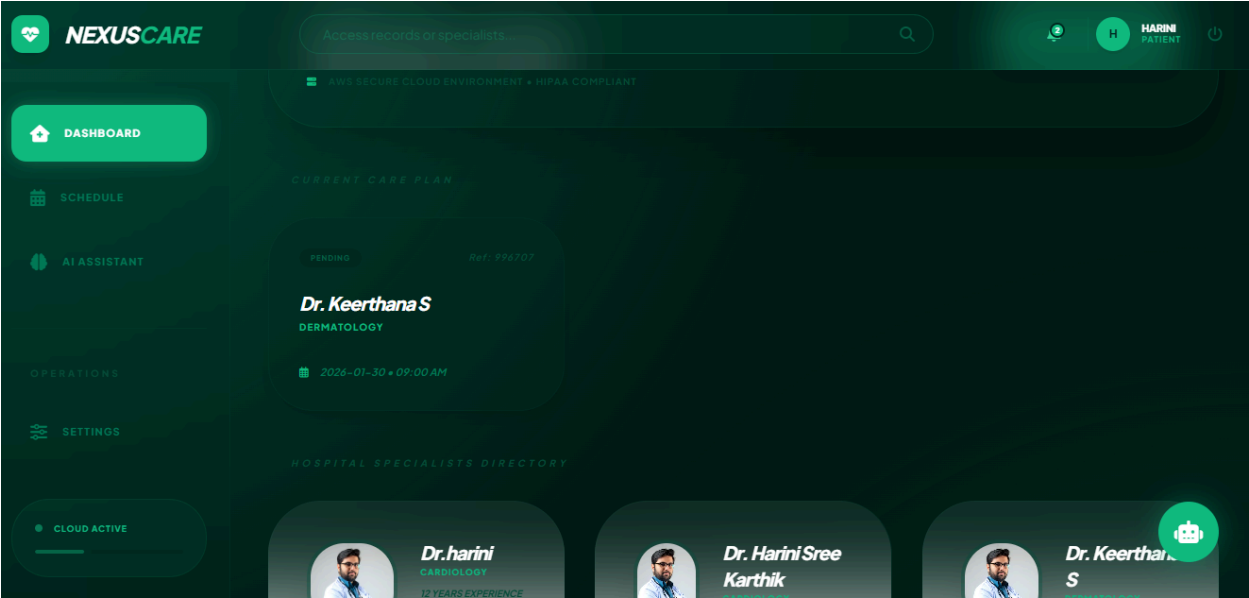
personal@email.com

SECURE PASKEY

AUTHENTICATE SESSION

NEW TO NEXUSCARE? [Provision Your Identity](#)

Home Page:



Booking Details:

SCHEDULE APPOINTMENT



SELECT YOUR PREFERRED CARE SESSION

APPOINTMENT DATE

dd-mm-yyyy



AVAILABLE TIME

Select a Slot



CONTACT PHONE NUMBER

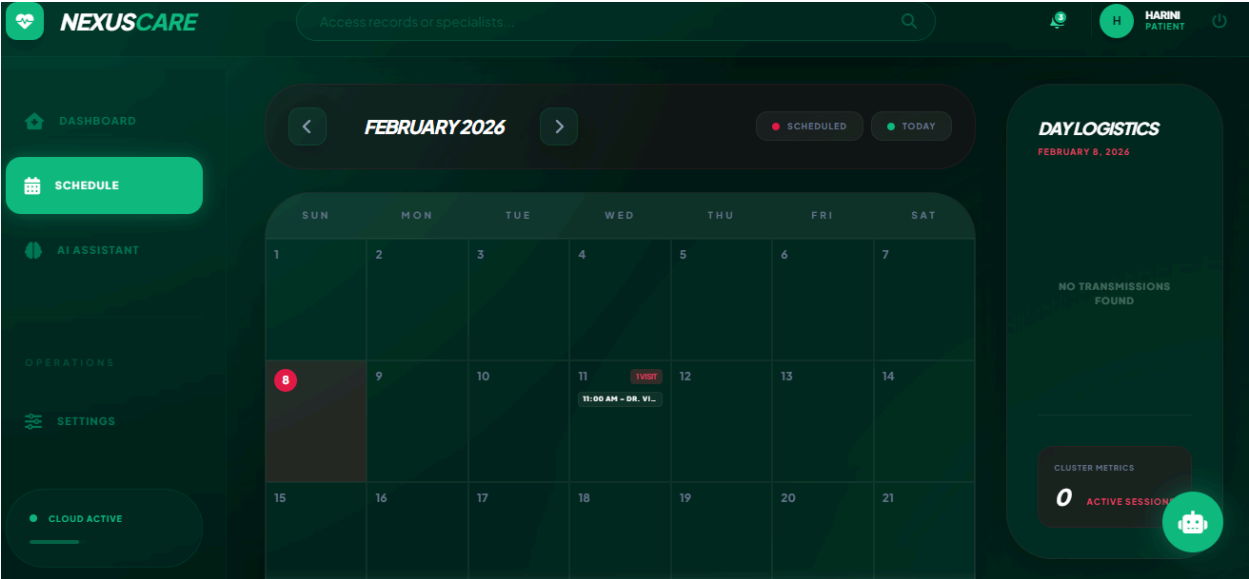
+1(555) 000-0000

CANCEL

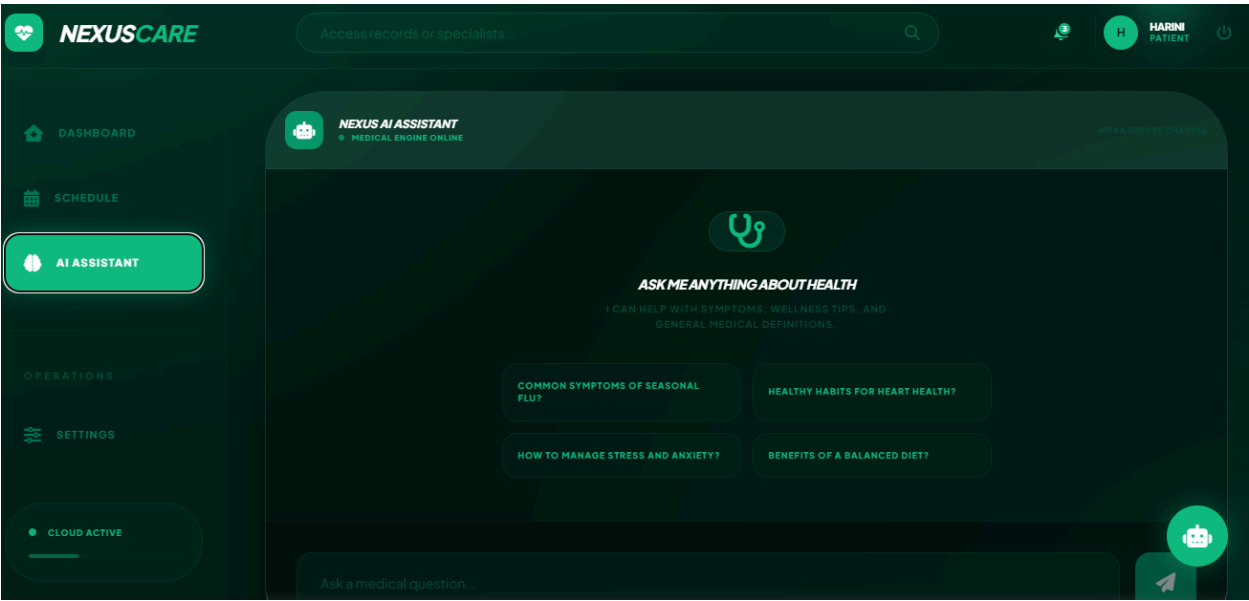
CONFIRM BOOKING

Booked Schedule:

Booked Schedule:



AI - Assistant:



Admin-Details Creation:

PATIENT

ADMINISTRATOR

LEGAL NAME

Admin Full Name

ADMINISTRATOR EMAIL ID

admin@nexus.io

SECURE PASKEY

SECURITY CHALLENGE

What was your first pet name? ▾

Secret answer...

Admin- Login:

IDENTITY PROTOCOL

PATIENT ADMINISTRATOR

ADMINISTRATOR EMAIL ID

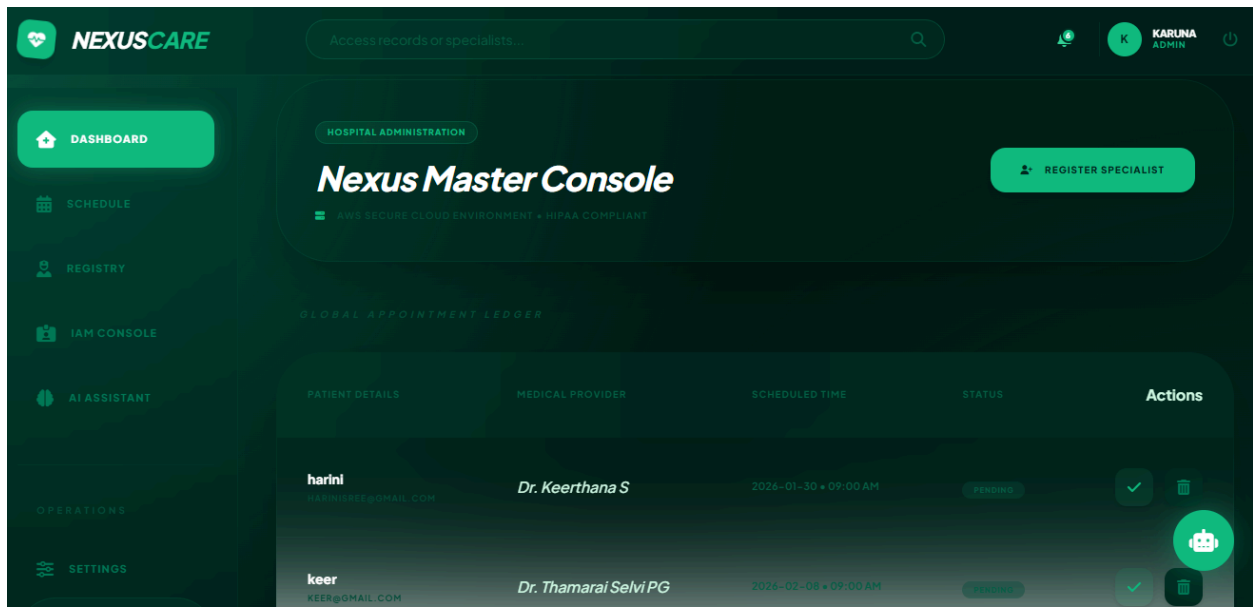
admin@nexus.io

SECURE PASSKEY

AUTHENTICATE SESSION

NEW TO NEXUSCARE? [Provision Your Identity](#)

Admin-Home Page:



Register Specialist:

REGISTER SPECIALIST

AUTHORIZED PERSONNEL ONLY

FULL LEGAL NAME

Dr. Aruna S

MEDICAL SPECIALTY

Dermatology

EXPERIENCE METRIC

25 Years

PROFILE IMAGE URL

<https://images.unsplash.com/pt>

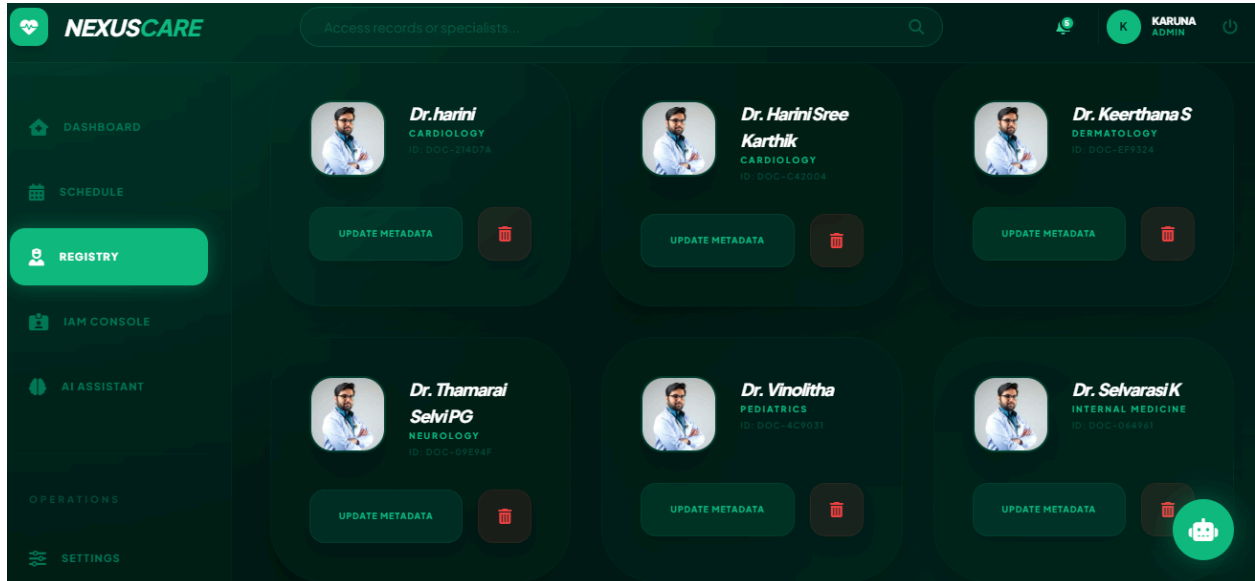
PROFESSIONAL BIO

Dr. Aruna S. is a dedicated and compassionate dermatologist specializing in the diagnosis and treatment of skin, hair, and nail disorders. With a strong foundation in clinical dermatology, Dr. [Name] is committed to providing patient-centered care using evidence-based medical practices and the latest advancements in dermatological science.

CANCEL

LINK REGISTRY RECORD

Registered- Specialist:



CONCLUSION

The SnapStream platform has been successfully implemented as a cloud-native media management and analysis solution powered by AWS services. By integrating scalable and serverless cloud components, the system delivers a secure, efficient, and intelligent framework for handling large volumes of multimedia content. This implementation demonstrates how modern cloud technologies can simplify complex workflows associated with media storage, processing, and analysis.

Through the seamless integration of Amazon S3, AWS Lambda, and AI/ML services such as Amazon Rekognition, Transcribe, and Comprehend, SnapStream automates time-intensive tasks like tagging, transcription, and content analysis. This not only enhances operational productivity but also empowers users with data-driven insights and faster access to valuable media information.

The platform's modular design ensures scalability and cost efficiency, while AWS IAM, DynamoDB, and SNS collectively guarantee security, reliability, and real-time communication. Extensive testing has validated all critical functionalities, from secure uploads to automated analysis pipelines, confirming SnapStream's readiness for production-level deployment.

In conclusion, SnapStream represents a significant step forward in intelligent media management, showcasing the power of AWS cloud infrastructure to transform traditional media workflows into an automated, scalable, and insight-driven ecosystem.